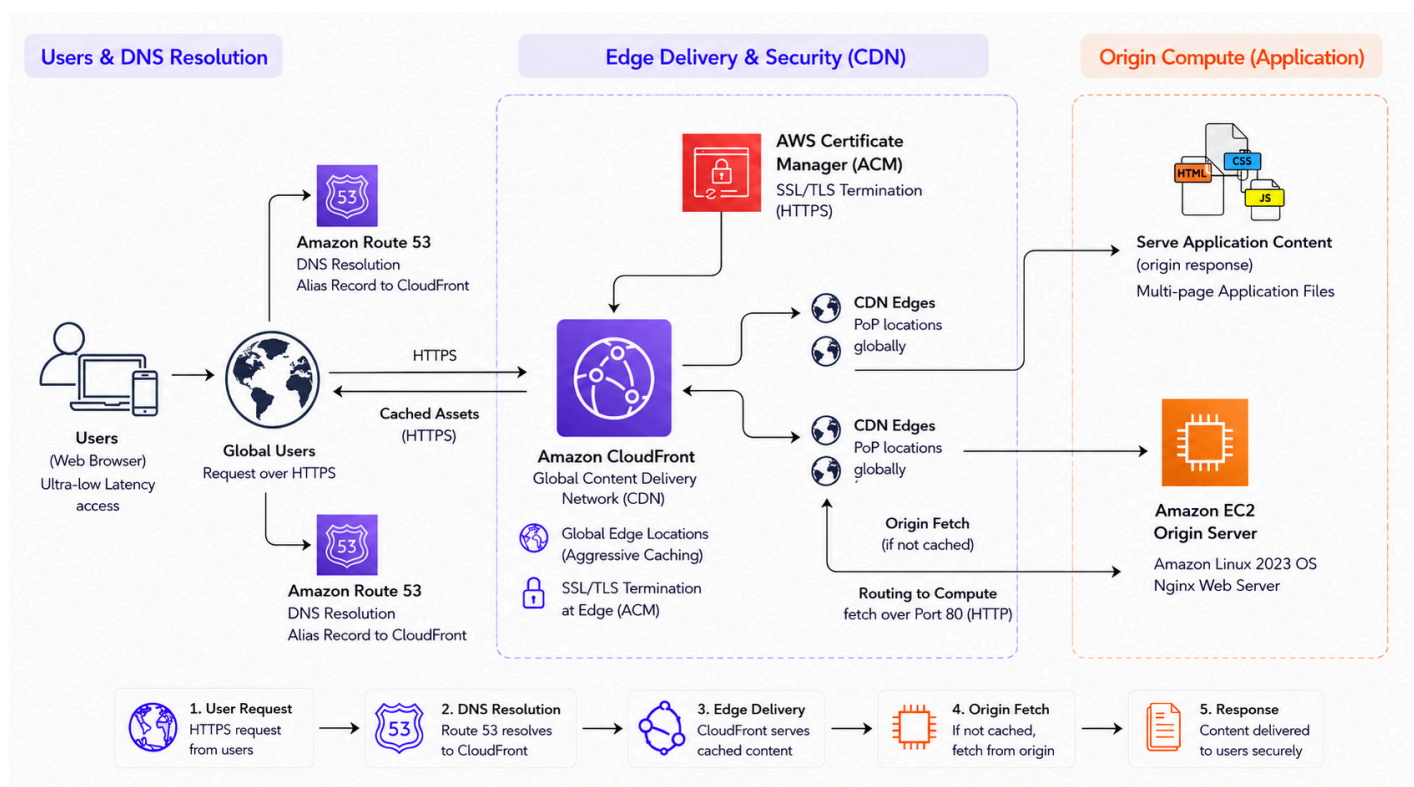


Global Content Distribution: Edge-Optimized Architecture

Objective: To architect and deploy a highly available, globally distributed delivery network for a multi-page frontend application, emphasizing edge caching, HTTPS enforcement, and strict origin security.

Solution: Implemented a decoupled content delivery architecture where a multi-page application (HTML/Tailwind CSS/JS) is hosted on an Amazon EC2 instance running Nginx server, set up as a CloudFront origin. Traffic is cached and routed globally via Amazon CloudFront, with custom DNS resolution managed by Amazon Route 53 and end-to-end TLS encryption provided by AWS Certificate Manager (ACM).



Phase 1: Compute & Web Server (Amazon EC2)

An EC2 instance (virtual server) is launched and configured to serve the frontend application files.

1) EC2 Instance Details:

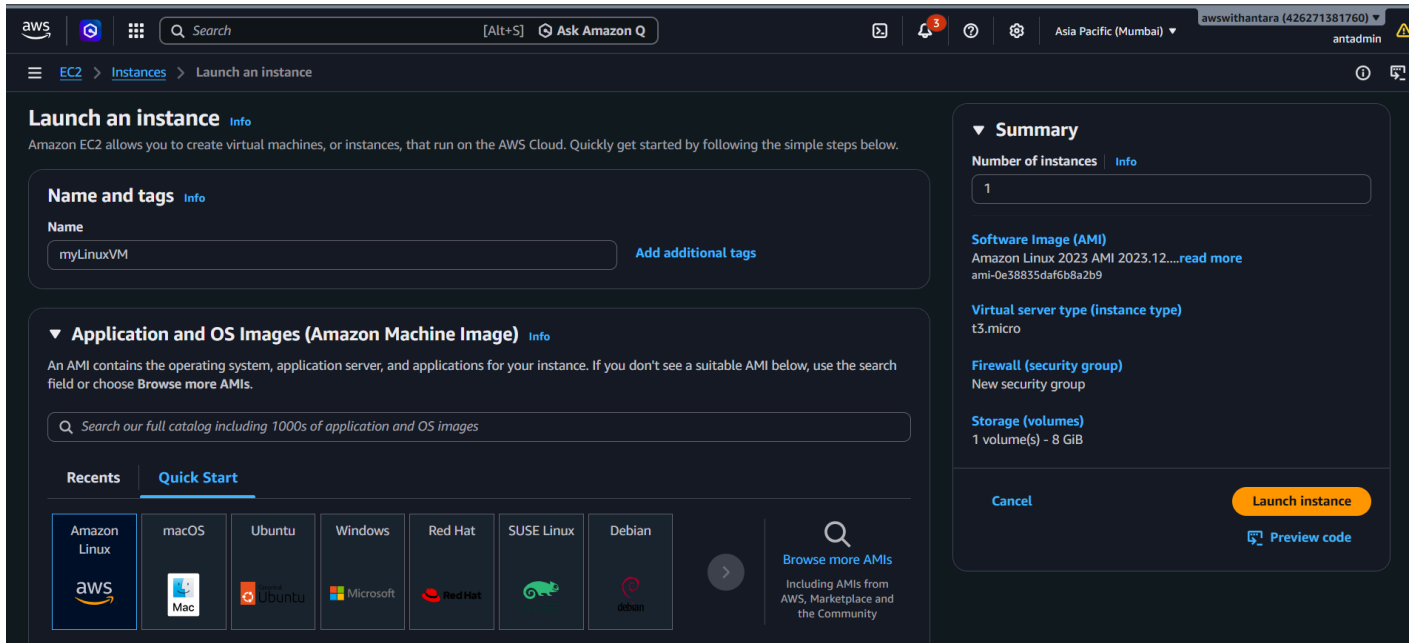
- Name: myLinuxVM
- Amazon Machine Image (AMI): Amazon Linux 2023 kernel 6.1 AMI (Free tier eligible)
- Instance type: t3.micro (Free tier eligible)
- Key-pair name: mumbai-key
- Security group name: EC2-CloudFront-Origin-SG (custom created security group)

- Security group description: my custom security group
- Inbound Rule 1 (**SSH**):
 - Port: 22
 - Source type: **My IP** (strictly limited to admin access)
- Inbound Rule 2 (**HTTP**):
 - Port: 80
 - Source type: **Custom**
 - Source: com.amazonaws.global.cloudfront.origin-facing (**pl-9aa247f3**)

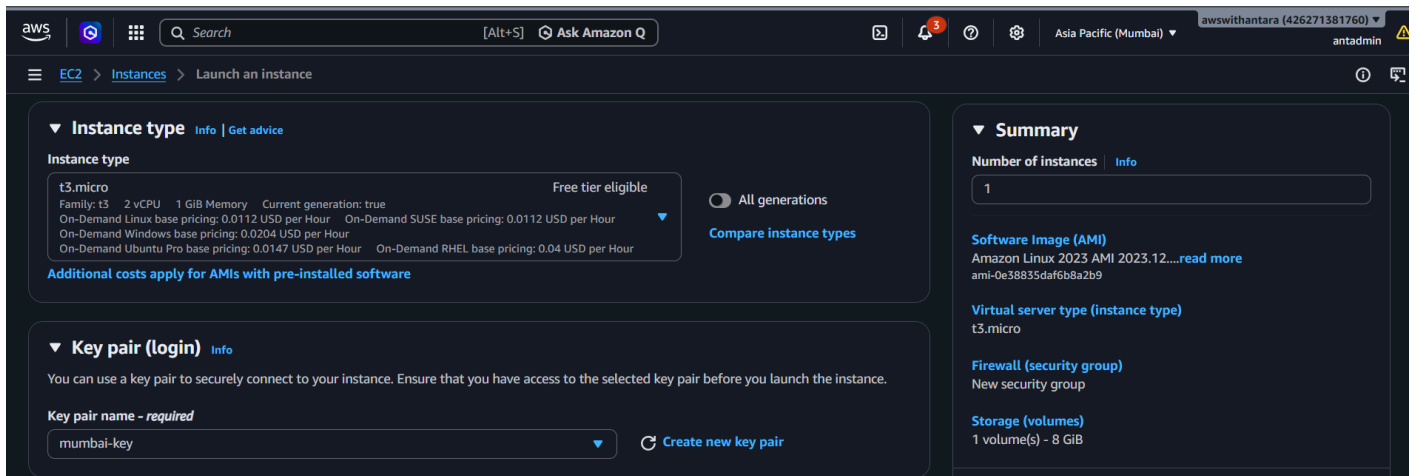
2) Application Deployment:

- EC2 instance (myLinuxVM) connected via SSH command using terminal
 - cd Downloads (location of the SSH Key)
 - ssh -i "mumbai-key.pem" ec2-user@ec2-65-0-102-31.ap-south-1.compute.amazonaws.com
- Linux commands to **install** and **enable** Nginx web server on **myLinuxVM**
 - sudo yum install nginx -y
 - sudo systemctl start nginx
 - sudo systemctl enable nginx
- Frontend files transferred from local machine to **myLinuxVM** using separate terminal
 - cd Downloads (location of the SSH Key and zipped Frontend files)
 - scp -i mumbai-key.pem Codespot_Project.zip ec2-user@65.0.102.31:/home/ec2-user
- Linux commands to deploy the application to **myLinuxVM** using the initial terminal where the web server is installed and enabled on the EC2 instance. These commands are used from the home directory of ec2-user, i.e., **/home/ec2-user**
 - ls (shows **Codespot_Project.zip** transferred to **/home/ec2-user**)
 - unzip **Codespot_Project.zip** (to unzip and extract files)
 - ls (shows both **Codespot_Project.zip** and **Codespot_Project** in **/home/ec2-user**)
 - sudo mv Codespot_Project/* /usr/share/nginx/html (to move all files from **/home/ec2-user** to **/usr/share/nginx/html**)

The **scp** command is used to securely transfer files from the local system to the EC2 instance. **/usr/share/nginx/html** is the location of the default html file in the Nginx web server.



EC2 Instance Launch: Name & Operating System (AMI)



EC2 Instance Launch: Instance Type & Key Pair

- Operating System (AMI) & Instance type of the EC2 instance (**myLinuxVM**) are within the AWS Free tier, thereby ensuring cost optimization
- SSH Key Pair (**mumbai-key**) is essential for connecting to **myLinuxVM**

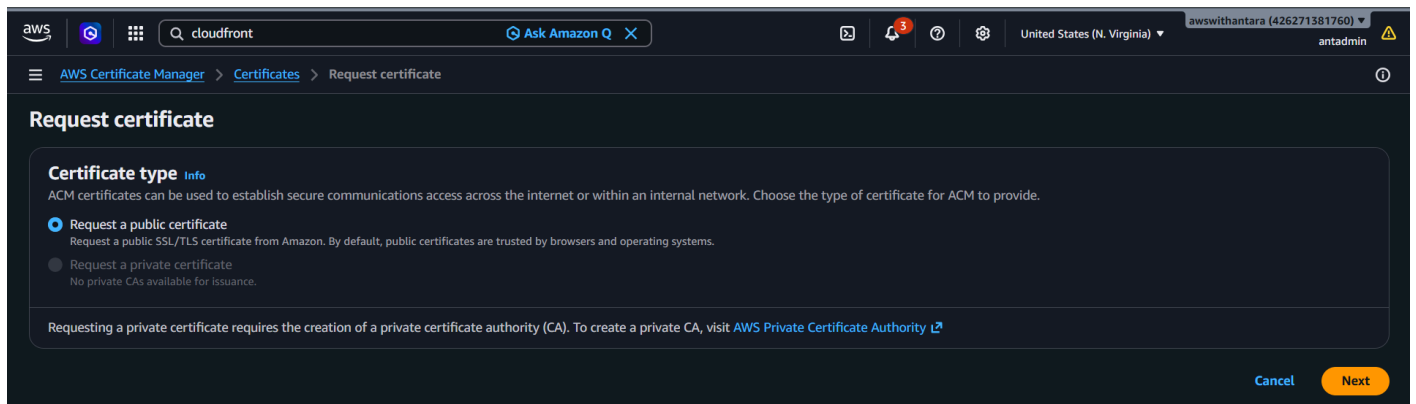
Phase 2: Security & Encryption (Amazon Certificate Manager)

ACM is mandatory for ensuring security and encryption for the custom domain.

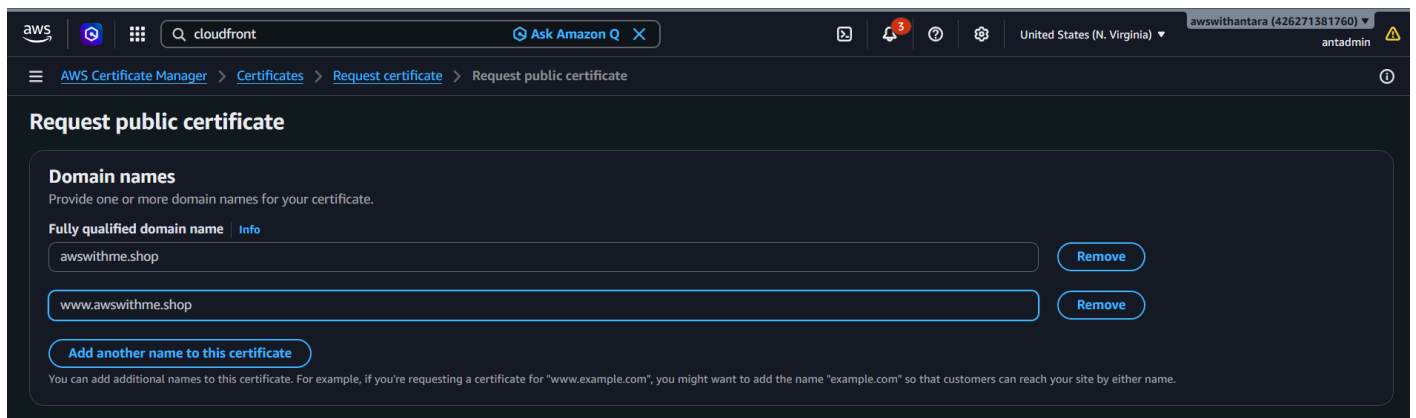
Certificate Details:

- Certificate type: Request a public certificate
- Domain names: awswithme.shop & www.awswithme.shop
- Validation method: DNS validation
- Key algorithm: RSA 2048 (default)

The certificate must be issued in **us-east-1** (North Virginia) because CloudFront's global edge network was designed to pull certificates exclusively from this region.



Issue Certificate in ACM: Certificate type



Issue Certificate in ACM: Domain Names

- Region is switched to **us-east-1 (N. Virginia)** in the ACM console since it's mandatory for CloudFront
- In addition to awswithme.shop, a subdomain is added to the certificate as required

aws cloudfront Ask Amazon Q United States (N. Virginia) awswithantara (426271381760) antadmin

AWS Certificate Manager > Certificates > Request certificate > Request public certificate

Validation method Info
Select a method for validating domain ownership.

☒ **DNS validation - recommended**
Choose this option if you are authorized to modify the DNS configuration for the domains in your certificate request.

☐ **Email validation**
Choose this option if you do not have permission or cannot obtain permission to modify the DNS configuration for the domains in your certificate request.

Key algorithm Info
Select an encryption algorithm. Some algorithms may not be supported by all AWS services.

☒ **RSA 2048**
RSA is the most widely used key type.

☐ **ECDSA P 256**
Equivalent in cryptographic strength to RSA 3072.

☐ **ECDSA P 384**
Equivalent in cryptographic strength to RSA 7680.

Issue Certificate in ACM: Validation Method & Key Algorithm

aws cloudfront Ask Amazon Q United States (N. Virginia) awswithantara (426271381760) antadmin

AWS Certificate Manager > Certificates > abcec57e-8b6f-46d7-ad18-192b5934d7f3 > Create DNS records in Amazon Route 53

Create DNS records in Amazon Route 53 (2/2)

Search domains 2 matches

Validation status = Pending validation or Validation status = Failed and Is domain in Route 53? = Yes Clear filters

Domain	Validation status	Is domain in Route 53?
awswithme.shop	Pending validation	Yes
www.awswithme.shop	Pending validation	Yes

Cancel Create records

Issue Certificate in ACM: DNS Records created in Route 53

aws Search [Alt+S] United States (N. Virginia) awswithantara (426271381760) antadmin

AWS Certificate Manager (ACM) < abcec57e-8b6f-46d7-ad18-192b5934d7f3 Delete

List certificates
Request certificate
Import certificate
AWS Private CA

Certificate status

Identifier
abcec57e-8b6f-46d7-ad18-192b5934d7f3

Status
Issued

ARN
arn:aws:acm:us-east-1:426271381760:certificate/abcec57e-8b6f-46d7-ad18-192b5934d7f3

Type
Amazon Issued

Domains (2) Create records in Route 53 Export to CSV

Domain	Status	Renewal status	Type	CNAME name
awswithme.shop	Success	-	CNAME	_ca0693e223d3629920f91965ee6b29a0.awswitl
www.awswithme.shop	Success	-	CNAME	_60c26264e7208c8649e8ddaaa929ac44.www.av chon

Issue Certificate in ACM: Certificate Status

Phase 3: Global Content Delivery (Amazon CloudFront)

A CloudFront Distribution is created with EC2 origin to speed up content delivery through global edge caching.

1) Distribution Details:

- Distribution name: my-cloudfront-origin-server
- Description: Origin Server for my web application
- Distribution type: Single website or app

2) Origin Details:

- Origin type: Other
- Custom origin: ec2-65-0-102-31.ap-south-1.compute.amazonaws.com (public DNS of the EC2 instance **myLinuxVM**)
- Origin settings: Customize origin settings
- Protocol: HTTP only
- Origin IP address type: IPv4-only
- Cache settings: Customize cache settings
- Viewer protocol policy: Redirect HTTP to HTTPS
- Allowed HTTP methods: GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE (default)
- Edit General Settings:
 - Alternate Domain Name (CNAME): awswithme.shop & www.awswithme.shop
 - Custom SSL certificate: awswithme.shop
(**abcec57e-8b6f-46d7-ad18-192b5934d7f3**)

The screenshot shows the AWS CloudFront console's 'Create distribution' wizard. The breadcrumb navigation at the top reads 'CloudFront > Distributions > Create distribution'. On the left, a vertical list of steps is shown: Step 1 'Get started' (selected with a blue circle), Step 2 'Specify origin', Step 3 'Enable security', Step 4 'Get TLS certificate', and Step 5 'Review and create'. The main content area is titled 'Get started' with a subtitle: 'Connect your websites, apps, files, video streams, and other content to CloudFront. We optimize the performance, reliability, and security for your web traffic.' Below this, the 'Distribution options' section is displayed. It includes a text input for 'Distribution name' containing 'my-cloudfront-origin-server', a text input for 'Description - optional' containing 'Origin Server for my web application', and two radio button options for 'Distribution type': 'Single website or app' (selected) and 'Multi-tenant architecture - New'. The 'Single website or app' option has a sub-note: 'Choose if each website or application will have a unique configuration.' The 'Multi-tenant architecture - New' option has a sub-note: 'Choose when you have multiple domains that need to share configurations. This is a common architecture for SaaS providers.'

CloudFront Distribution Setup: Distribution Name, Description & Type

Step 1: Get started

Step 2: **Specify origin**

Step 3: Enable security

Step 4: Review and create

Specify origin

Your origin is where your content (such as a website or app) lives. CloudFront works with AWS-based origins and origins hosted on other cloud providers.

Origin type

- ☐ Amazon S3: Deliver static assets like files and images, statically generated websites or single page applications (SPA).
- ☐ Elastic Load Balancer: Deliver applications hosted behind ELB such as dynamic websites, web services, and APIs.
- ☐ API Gateway: Deliver API endpoints for REST APIs hosted on API Gateway.
- ☐ Elemental MediaPackage: Deliver end-to-end live events or video on demand (VOD).
- ☐ VPC origin: Deliver applications and content hosted within private VPCs, such as EC2 instances and Application Load Balancers.
- ☒ **Other**: Refer to any AWS or non-AWS origin through its publicly resolvable URL.

Origin

Custom origin
Choose an AWS origin, or enter your origin's domain name. [Learn more](#)

ec2-65-0-102-31.ap-south-1.compute.amazonaws.com

Origin path - optional
The directory path within your origin where your content is stored. [Learn more](#)

/path

CloudFront Distribution Setup: Origin Type & Domain (EC2 public DNS)

Step 1: Get started

Step 2: Specify origin

Step 3: **Origin settings**

Step 4: Review and create

Origin settings

Origin settings control how CloudFront connects to the specified origin.

☐ Use recommended origin settings

☒ **Customize origin settings**

☐ **Enable origin mutual TLS**
Securely validate the connection between your origin and the CloudFront distribution.

Add custom header - optional
CloudFront includes this header in all requests that it sends to your origin.

[Add header](#)

Enable Origin Shield
Origin shield is an additional caching layer that can help reduce the load on your origin and help protect its availability.

☒ **No**

☐ Yes

Protocol [Info](#)

☒ **HTTP only**

☐ HTTPS only

☐ Match viewer

HTTP port
Enter your origin's HTTP port. The default is port 80.

80

CloudFront Distribution Setup: Origin Protocol

- For Origin type, Other needs to be selected for EC2 as origin where public DNS of **myLinuxVM** is used as custom origin
- Protocol is selected as **HTTP only** since CloudFront connects to **myLinuxVM** over port 80

Origin IP address type [Info](#)
This setting determines which IP protocol(s) CloudFront uses when connecting to your origin.

- ☒ **IPv4-only**
Choose if your origin only supports IPv4 addresses.
- ☐ **IPv6-only**
Choose if your origin only supports IPv6 addresses.
- ☐ **Dualstack (IPv4 and IPv6)**
Choose if your origin supports both protocols. This helps to optimize reliability.

Cache settings
Cache settings determine when CloudFront serves cached content and when it fetches new content from the origin.

☐ Use recommended cache settings tailored to serving Custom origin content

☒ **Customize cache settings**

Viewer protocol policy

- ☐ HTTP and HTTPS
- ☒ **Redirect HTTP to HTTPS**
- ☐ HTTPS only

Allowed HTTP methods

- ☐ GET, HEAD
- ☐ GET, HEAD, OPTIONS
- ☒ **GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE**

Cache HTTP methods
GET and HEAD methods are cached by default.

- ☐ **OPTIONS**

CloudFront Distribution Setup: Origin IP Address Type, Viewer Protocol Policy & HTTP Methods

General

Description - optional
Origin Server for my web application

Alternate domain name (CNAME) - optional
Add the custom domain names that you use in URLs for the files served by this distribution.

awsithme.shop [Remove](#)

www.awswithme.shop [Remove](#)

[Add item](#)

To add a list of items, use the [bulk editor](#).

Custom SSL certificate - optional
Associate a certificate from AWS Certificate Manager. The certificate must be in the US East (N. Virginia) Region (us-east-1).

awsithme.shop (abcec57e-8b6f-46d7-ad18-192b5934d7f3) [Request certificate](#)

☐ Legacy clients support - \$600/month prorated charge applies. Most customers do not need this.
CloudFront allocates dedicated IP addresses at each CloudFront edge location to serve your content over HTTPS.

Security policy
The security policy determines the SSL or TLS protocol and the specific ciphers that CloudFront uses for HTTPS connections with viewers (clients).

☒ **TLSv1.3_2025**

CloudFront Distribution Setup: Alternate Domain Names & Custom SSL Certificate

- The allowed HTTP methods are selected as **GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE** which is sufficient for static sites
- Custom SSL certificate** must be added along with alternate domain names for SSL/TLS handshake

Phase 4: DNS Resolution (Amazon Route 53)

DNS records are created in the public hosted zone of the custom domain (**awswithme.shop**) in order to connect it to CloudFront DNS.

DNS Records Details:

- Record 1:
 - Record name: awswithme.shop
 - Record type: A
 - Route traffic to: Alias to CloudFront distribution
 - Routing policy: Simple routing
- Record 2:
 - Record name: www.awswithme.shop
 - Record type: A
 - Route traffic to: Alias to CloudFront distribution
 - Routing policy: Simple routing

The screenshot shows the AWS Route 53 console's 'Create record' wizard. The breadcrumb trail at the top indicates the path: Route 53 > Hosted zones > awswithme.shop > Create record. The main heading is 'Create record' with an 'Info' link. Below this, there's a 'Quick create record' section with a 'Switch to wizard' link. Under 'Record 1', there's a 'Record name' field with 'subdomain' as a placeholder and 'awswithme.shop' as the value. To the right, the 'Record type' is set to 'A - Routes traffic to an IPv4 address and some AWS resources'. Below the record name, there's a note: 'Keep blank to create a record for the root domain.' The 'Alias' toggle is turned on. Under 'Route traffic to', the dropdown is set to 'Alias to CloudFront distribution', and the region is 'US East (N. Virginia)'. Below this, there's a note: 'An alias to a CloudFront distribution and another record in the same hosted zone are global and available only in US East (N. Virginia).' and a search bar containing 'd16mfhdvbmjsi.cloudfront.net'. The 'Routing policy' is set to 'Simple routing'. There's an 'Evaluate target health' toggle set to 'No'. At the bottom right, there's an 'Add another record' button.

DNS Records Creation in Route 53: Record Details for **awswithme.shop**

The screenshot shows the AWS Route 53 console's 'Create record' page. The breadcrumb navigation at the top indicates the path: Route 53 > Hosted zones > awswithme.shop > Create record. The page title is 'Create record' with an 'Info' link. Below the title is a 'Quick create record' section with a 'Switch to wizard' link. A 'Record 1' section contains the following fields: 'Record name' (www) and 'Record type' (A - Routes traffic to an IPv4 address and some AWS resources). Below these is a note: 'Keep blank to create a record for the root domain.' The 'Alias' checkbox is checked. The 'Route traffic to' dropdown is set to 'Alias to CloudFront distribution', with a sub-selection of 'US East (N. Virginia)'. Below this is a note: 'An alias to a CloudFront distribution and another record in the same hosted zone are global and available only in US East (N. Virginia)'. The 'Routing policy' dropdown is set to 'Simple routing'. The 'Evaluate target health' checkbox is unchecked. At the bottom right is an 'Add another record' button.

DNS Records Creation in Route 53: Record Details for **www.awswithme.shop**

Inference: The implementation of this edge-optimized architecture demonstrates a robust global content delivery. By successfully decoupling domain resolution (Route 53), edge caching (CloudFront), and origin compute (EC2), this architecture establishes a scalable foundation capable of handling enterprise-level traffic while maintaining stringent security postures.

- **Key Engineering Decisions:**

- **Edge Caching (CloudFront):** Implemented to drastically reduce latency by caching static assets at global Edge Locations, minimizing the compute load on the EC2 origin.
- **Origin Protection (Prefix Lists):** Configured the EC2 Security Group to exclusively accept incoming HTTP traffic from the AWS Managed Prefix List for CloudFront. This prevents unwanted bypassing of the CDN security policies via direct IP routing.
- **SSL Termination:** Terminated the TLS connection at the CloudFront edge using AWS ACM, offloading cryptographic processing from the EC2 compute instance while ensuring end-to-end encryption for the user.